arXiv:2509.24125v3 [cs.LG] 10 Dec 2025

The Impossibility of Inverse Permutation Learning in Transformer Models

Rohan Alur*
Bridgewater AIA Labs
Massachusetts Institute of Technology
ralur@mit.edu

Chris Hays*
Massachusetts Institute of Technology
jhays@mit.edu

Manish Raghavan
Massachusetts Institute of Technology
mragh@mit.edu

Devavrat Shah
Massachusetts Institute of Technology
devavrat@mit.edu

Abstract

In this technical note, we study the problem of inverse permutation learning in decoder-only transformers. Given a permutation and a string to which that permutation has been applied, the model is tasked with producing the original ("canonical") string. We argue that this task models a natural robustness property across a variety of reasoning tasks, including long-context retrieval, multiple choice QA and in-context learning. Our primary contribution is an impossibility result: we show that an arbitrary depth, decoder-only transformer cannot learn this task. This result concerns the expressive capacity of decoder-only transformer models and is agnostic to training dynamics or sample complexity. We give a pair of alternative constructions under which inverse permutation learning is feasible. The first of these highlights the fundamental role of the causal attention mask, and reveals a gap between the expressivity of encoder-decoder transformers and the more popular decoder-only architecture. The latter result is more surprising: we show that simply padding the input with "scratch tokens" yields a construction under which inverse permutation learning is possible. We conjecture that this may suggest an alternative mechanism by which chain-of-thought prompting or, more generally, intermediate "thinking" tokens can enable reasoning in large language models, even when these tokens encode no meaningful semantic information (e.g., the results of intermediate computations).

1 Introduction

One of the most striking features of modern large language models (LLMs) is the emergence of general-purpose reasoning abilities at scale. Even relatively early LLMs were capable of in-context learning [5], multiple choice question answering [15], long-context reasoning [20], and deductive logical reasoning tasks [14]. It was not obvious, a priori, that a single model could be trained to handle such a diverse range of tasks, since each might appear to demand distinct—and potentially incompatible—architectures, training recipes or inductive biases.

*These authors contributed equally to this work

In this work, we focus on a particular inductive bias: permutation invariance. Modern LLMs typically encode positional information through positional encodings [31]. This is, of course, a desirable property for a language model, as the order of words is inextricably linked to their meaning. However, this sensitivity to ordering can present challenges in other contexts. For example, consider in-context learning: given k labeled examples, a model is asked to predict the label for a final unlabeled input. It is typically desirable that these predictions are *invariant* to ordering of the examples. In multiple choice question answering (QA), we'd similarly like invariance to the order of answer options; in long-context reasoning, we might want invariance to the order of in-context facts; in deductive reasoning tasks a model should be invariant to the ordering of logical predicates; in generative verification tasks, the verifier should be invariant to the ordering of candidate solutions.

Unfortunately, modern LLMs fail to satisfy any of these desiderata [25, 6, 21, 27, 22, 19]. This sensitivity to ordering manifests as seemingly arbitrary failures, undermining reliability and, ultimately, trust in these models. This problem is well-studied, with proposed solutions including altering the training loss to encourage permutation invariance [7], optimizing input ordering [13, 4, 3], and modifications to the architecture [9, 34, 11]. However, none of these solutions address the fundamental question: can a standard decoder-transformer *learn* to guarantee permutation invariance?

Inverse permutation learning. We observe that *inverse permutation learning* is a sufficient and natural manner in which to guarantee order invariance. For example, given a sequence of (key, value) pairs as in-context examples, it might first map these examples to a lexicographic ordering over keys, guaranteeing invariance to the order in which they are presented. We model this task as *inverse permutation learning*: given a permuted sequence and a description of the permutation which was applied, the model should learn to output a "canonical" version of the sequence.

Permutation	**Permuted Sequence**		**Canonical Ordering**
[2 0 1]	[♣ ♡ ♠]	$\xrightarrow{\text{apply inverse}}$	[♡ ♠ ♣]
[1 2 0]	[♠ ♣ ♡]	$\xrightarrow{\text{apply inverse}}$	[♡ ♠ ♣]

Figure 1: Visualizing inverse permutation learning. The model is given a permutation and a permuted sequence as input, and is tasked with inverting the permutation to recover the canonical ordering.

This task is perhaps easier than our motivating examples, as the model is given both a description of the permutation to be "undone" as well as the permuted sequence. Nonetheless, we show that a simplified transformer model cannot learn to complete this task.

Contributions. In Section 2.1, we derive an impossibility result for inverse permutation learning with a decoder-only, attention-only transformer architecture. The intuition underpinning this result is straightforward: all permutations (except the trivial identity permutation) require shifting at least one element from a later position in the sequence to an earlier position in the sequence. However, the structure of the causal attention mask prevents the necessary information transfer from later tokens to earlier ones.

We complement this impossibility result with a pair of existence proofs in Section 2.2 and Section 2.3. In Section 2.2, we demonstrate that removing causal masking to allow for general two-way attention mechanisms yields a construction which solves the inverse permutation learning problem. This result suggests that alternative architectures, particularly encoder-decoder architectures, may not have the same limitations as the decoder-only architecture which underpins the most popular modern LLMs. We corroborate this result with an empirical analysis in simplified transformer model, which

demonstrates that accuracy in the inverse permutation learning task improves from the level of random guessing to nearly perfect accuracy if the causal attention mask is removed.[1] We also conduct a mechanistic analysis of the learned weights, which constitute an alternative constructive proof that removing causal attention masking admits a parameter configuration which solves the inverse permutation learning problem.

In Section 2.3 we show, more surprisingly, that *padding the input* with additional "scratch tokens" also yields a construction for inverse permutation learning. As our construction illustrates, this transformation of the input provides the model with "scratch space" which can be used to perform additional computation during the forward pass. We conjecture that this mechanism may partially explain the success of "chain-of-thought" prompting [32], scratchpad prompting [24] or the use of "reasoning tokens" [8], which perform a similar function by allowing the model to perform additional computation *whether or not the intermediate tokens encode meaningful semantic information about the problem* (e.g., the output of intermediate reasoning steps). This suggests a concrete and, to our knowledge, unexplored mechanism for enabling reasoning in transformer models.[2]

Limitations. We study the stylized transformer first proposed by Nichani et al. [23]. Our model preserves the most important features of the architecture, but omits multi-layer perceptron blocks, multi-head attention, and nontrivial positional embeddings, among other complexities. Our results also assume a natural but particular input representation. Furthermore, our constructive results concern the expressive capacity of transformers, but do not examine training dynamics or sample complexity required to achieve these parameter configurations.

Organization. We present formal statements of our technical results in Section 2. We discuss additional related work in Section A. We provide empirical results and conduct a mechanistic analysis to corroborate our primary contributions in Section B. All proofs are provided in Section C.

2 Statement of Technical Results

We begin with the definition of the inverse permutation function.

Definition 1 (Inverse permutation function)**.** *For fixed n, a permutation matrix $P \in \{0,1\}^{n\times n}$ is defined such that ith row represents the elementary unit vector $e_{\pi(i)}$ for some permutation $\pi(\cdot)$. For any n-row matrix Y_P, the inverse permutation function outputs $Y = P^{-1}Y_P$.*

We interpret each row of Y_P as corresponding to the embedding of a single token in the (permuted) input, and each row of Y as corresponding to the embedding of a single token of the (canonical) output. Observe that one function satisfying Definition 1 is the trivial linear transformation $(P, Y_P) \mapsto P^\top Y_P$, since $P^{-1} = P^\top$. As mentioned above, this task is arguably simpler than the more realistic tasks which motivate it: the description of the permutation P is given explicitly, rather than being determined by some function of the permuted input Y_P (e.g., its lexicographic index). Thus, we have eliminated the difficulty of determining the order in which inputs are arranged or the choice of canonical ordering during internal model computations. Similarly, by presenting Y_P as a matrix, we

[1] Our code is available here: https://github.com/johnchrishays/icl.

[2] Perhaps the closest related work is that of Abbe et al. [1], who propose the notion of an "agnostic" (i.e., unsupervised) scratchpad. In this setting, the model may first produce a fixed number of output tokens which do not appear in the training loss for an otherwise standard supervised learning problem. Abbe et al. [1] conjecture that agnostic scratchpads are incompatible with sample efficient learning. Our results instead suggest padding the *input* with scratch tokens, but can nonetheless be interpreted as evidence which contradicts this conjecture, at least in settings where scratch tokens can help circumvent the constraints imposed by causal attention masking.